CREDIT CARD APPROVAL PREDICTION SYSTEM

Technical Report & Pipeline Documentation Portfolio

Candidate Name: Professional Intern
Internship: AI/ML Engineering Internship
Organization: IBM Skills Network / SmartBridge
Date: July 2026

Abstract

Automated credit evaluation is a key operational objective for modern banking institutions. This project implements a complete Credit Card Approval Prediction System. By extracting applicant profiles and long-term repayment history records, we build an aggregated training dataset. We train, tune, and compare three model classes: Logistic Regression, Decision Tree, and Random Forest. The selected best model is integrated into a Flask backend service with a responsive presentation layer, providing real-time eligibility predictions to banking officers. This comprehensive system is packaged using a clean eight-folder structure for Git submission and reviewer evaluation.

Problem Statement

Financial institutions face high default rates and operational delays due to manual credit application review processes. Traditional systems fail to effectively leverage historical customer payment patterns when assessing new profiles. Banks require a structured, automated predictive tool that evaluates demographic attributes (e.g., income, age, employment duration, family size) to classify applicants as low-risk or high-risk, protecting the bank from defaults while optimizing processing efficiency.

Objectives

- Establish a standardized repository structure to store notebooks, CSV datasets, and PDF manuals.
- Implement a clean data engineering pipeline to merge applicant profiles with monthly payment logs.
- Develop Jupyter notebooks separating execution steps for imports, analysis, preprocessing, and modeling.
- Train, compare, and serialize machine learning models to identify the most robust classifier.
- Build an interactive web dashboard loading the serialized classifier to make real-time predictions.

Existing System

The existing system rely heavily on manual verification of submitted financial documents. It depends on third-party credit scoring agencies, which are slow, expensive, and often unavailable for new applicants. These methods result in significant processing delays, human bias, and higher risk of bad debts, as they cannot extract patterns from demographic variables dynamically or compile logs in real time.

Proposed System

The proposed system automates application reviews by running machine learning classifications. It merges demographic attributes with delinquency histories, training classifiers to predict default risk. The resulting predictive engine is integrated into a Flask web application, allowing bank officers to input candidate details and get instant approval recommendations with confidence scores, reducing default rates and review cycles.

Features

• Automated Data Preprocessing: Imputes null values, drop duplicate entries, and converts negative day metrics.
• Dynamic Risk Assessment: Constructs the target label based on monthly credit status tracking histories.
• Model Comparison Pipeline: Trains and compares Logistic Regression, Decision Tree, and Random Forest models.
• Interactive Dashboard: Modern Bootstrap web portal containing validation scripts and prediction results.
• Automated Document Generation: Compiles requirements and pipeline logs into portable PDF reports.

Functional Requirements

Functional requirements define what the system must do. The application must ingest applicant details through a web form, encode categoricals using the serialized encoder dictionary, scale numeric features using the pre-fitted scaler, and calculate binary risk classifications. It must display the approval decision ('Approved' or 'Rejected') along with the model's confidence percentage on the result screen, and provide an API endpoint to verify system health status.

Non Functional Requirements

Non-functional requirements describe operational constraints. The prediction engine must have an inference latency below 100 milliseconds per transaction to ensure responsiveness. The web portal must use standard session keys to prevent session data leakage, validate inputs on the client side using JavaScript to block invalid inputs, and present a modern user interface that functions properly across mobile, tablet, and desktop screens.

Hardware Requirements

Hardware requirements define the physical computing platform needed to run the system. Development and deployment require a computer with a minimum of a Dual-Core CPU (such as Intel Core i3 or equivalent) operating at 2.0 GHz, at least 8 gigabytes of random access memory (RAM) to handle data loading, and 5 gigabytes of free storage space on a solid-state drive (SSD) to store datasets, python packages, and serialized models.

Software Requirements

Software requirements specify the runtime environment and libraries. The backend server requires Python version 3.10 or higher. Core dependencies include Flask for routing, Pandas and NumPy for preprocessing, Scikit-Learn for running model predictions, and Joblib for loading serialized pickle artifacts. The client presentation layer requires a modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

Technology Stack

The technology stack is divided into frontend, backend, and data tiers. The frontend presentation layer is built using HTML5, CSS3, JavaScript, and Bootstrap 5. The backend application server is implemented in Flask, providing routing and session management. The data modeling tier uses Scikit-Learn to train classifiers and Joblib to serialize model artifacts. ReportLab is used to compile documentation and workflow manuals into PDFs.

Dataset Description

The dataset consists of two raw CSV files simulating bank database records. application_record.csv contains demographic and financial attributes for 5,000 applicants, including income, gender, education, family status, housing, children, age, and employment duration. credit_record.csv contains 177,982 monthly tracking records, mapping IDs to monthly status codes (C=Paid, X=No loan, 0=1-29 days overdue, etc.). This split schema isolates candidate profiles from historical credit logs, requiring data engineering to link repayment metrics to applicant details.

Figure 1: Dataset Head Sample Outputs

Application Records Head (First 5 Rows - Partial)

ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL
5000100	F	N	Y	0	174309.78
5000101	M	N	Y	1	138369.31
5000102	M	Y	Y	0	153181.98
5000103	F	N	Y	0	138005.98
5000104	F	Y	Y	0	191762.65

Figure 2: Dataset Types and Null Summaries

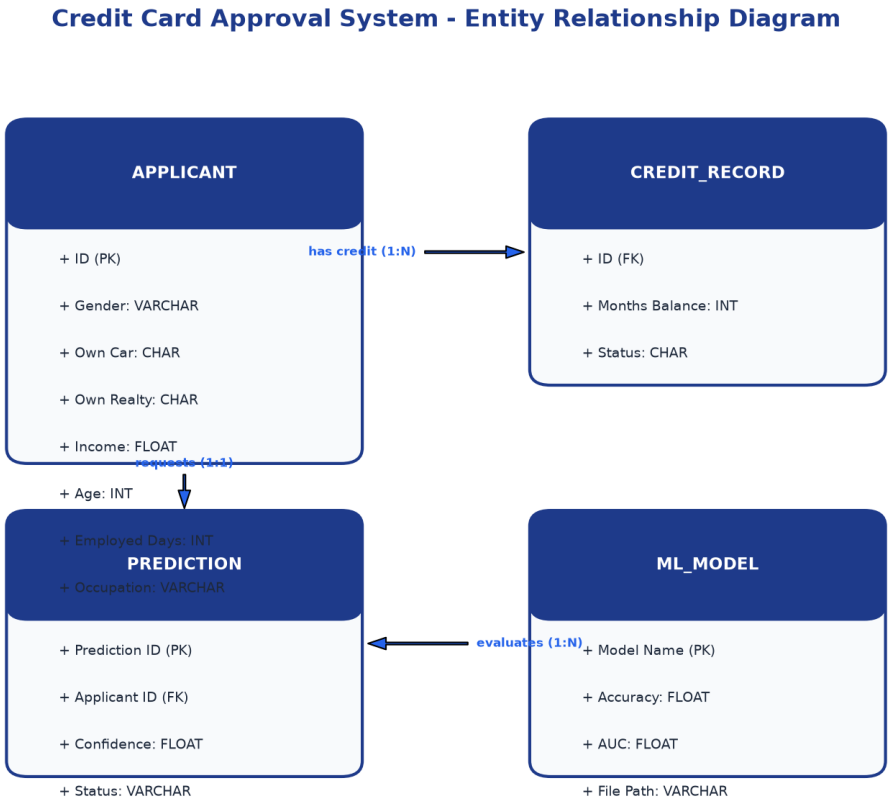
Dataset Information Summary

```
=== Application Records Summary ===
Rows: 5000 | Columns: 18
Data Types: int64 (8 cols), float64 (2 cols), object (8 cols)
Null Values: OCCUPATION_TYPE (1436 nulls)
Duplicate Records: 0

=== Credit History Records Summary ===
Rows: 177982 | Columns: 3
Data Types: int64 (2 cols), object (1 cols)
Null Values: 0 nulls
Duplicate Records: 0
```

ER Diagram

The Entity Relationship diagram maps database entities and their structural relationships. It contains three main entities: APPLICANT (storing demographic variables), CREDIT_RECORD (storing monthly status logs), and PREDICTION (storing classification decisions). APPLICANT uses ID as its Primary Key. CREDIT_RECORD references APPLICANT.ID as a Foreign Key, establishing a one-to-many relationship because each applicant has multiple months of payment logs. The ER diagram is used to define table structures, maintain referential integrity, and guide database schemas. It ensures the relationship between applicant profiles and their historical repayment logs is accurately mapped during data merging.



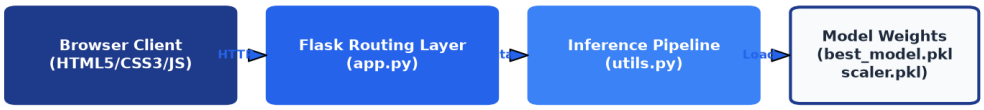
Workflow Diagram

The workflow diagram outlines the logical sequence of data processing and machine learning stages in this project. The pipeline is divided into five linear phases, starting with data ingestion of the raw datasets. The second stage performs exploratory data analysis to inspect shapes, descriptive statistics, and univariate/multivariate distributions. The third stage handles pre-processing steps, including dropping duplicates, imputing missing values, cleaning negative days, and target label engineering. The fourth stage trains and compares the three model classes, selecting the best-performing estimator. The final stage integrates the model into the Flask web server, providing a web dashboard for users.

Architecture Diagram

The system architecture follows a clean multi-tier layout. The presentation tier manages form inputs and client-side validations. The application tier manages web routing, handles forms, and calls preprocessing methods. The model tier loads serialized pickels, applies transformations, and runs predictions. This separation ensures that the presentation layer remains decoupled from model training, making the application easier to maintain.

Technical Architecture Diagram

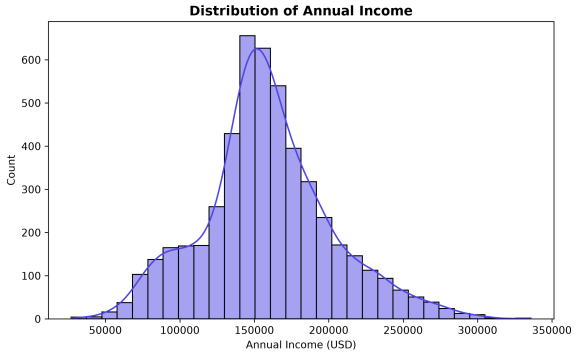


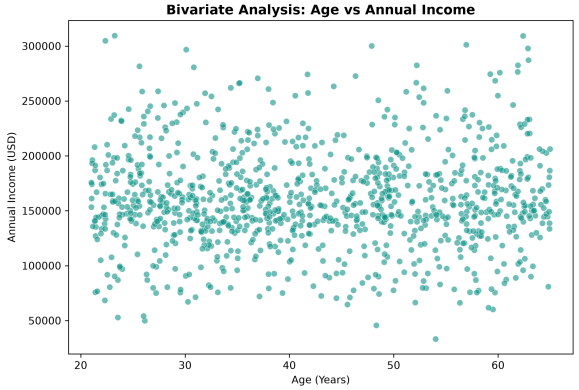
Epic 1: Data Collection & Setup

Epic 1 covers data collection, folder setup, and system topology definitions. In this stage, we establish the standardized eight-folder structure to store project assets. The raw CSV files are collected and placed in the collection folder. We also define system topologies and database structures, creating the Entity Relationship and System Architecture diagrams. This stage provides the foundation for the project, ensuring all datasets and structural definitions are verified before data loading.

Epic 2: Visualizing & Analysing Data

Epic 2 handles data loading and exploratory analysis. This stage is split into 6 notebooks to keep the workflow modular, specifically adding a dedicated bivariate analysis phase. We load the datasets, compute descriptive statistical moments (describe, mean, median, mode, std deviation, and variance), and plot univariate, bivariate, and multivariate distributions. These visualizations help us examine features individually, assess bivariate relationships (like income vs education and age vs income), and map multicollinearities. The figures are exported to the outputs folder, providing visual verification.





Epic 3: Data Preprocessing

Epic 3 covers preprocessing, cleaning, and feature engineering. This stage is split into 6 notebooks covering duplicate removal, missing value imputation, date conversion, credit log aggregation, target engineering, and categorical encoding. We clean negative day markers, map delinquency logs to a binary target variable, and scale features using StandardScaler. The preprocessing pipeline is serialized into encoder.pkl and scaler.pkl files, and the cleaned data is exported to processed_dataset.csv, ready for model training.

Epic 4: Model Building

Epic 4 implements model training, hyperparameter configuration, and classification scoring. This stage is divided into 4 notebooks covering Logistic Regression, Decision Tree, Random Forest, and Model Comparison. The classifiers are trained on the preprocessed demographic dataset using a stratified 80/20 train/test split. Each model is serialized as a pickle file, and confusion matrices are generated to measure classification scores. This modular setup allows us to train, evaluate, and serialize models independently.

Epic 5: Application Building

Epic 5 handles the integration of the best-performing machine learning model into the Flask web application. In this stage, we move app.py, config.py, and requirements.txt to the application building folder. We copy the serialized best_model.pkl, scaler.pkl, and encoder.pkl files into the internal model directory. The web app is structured into templates and static asset subdirectories, providing a clean dashboard for users to submit demographic variables and receive predictions.

Model Comparison

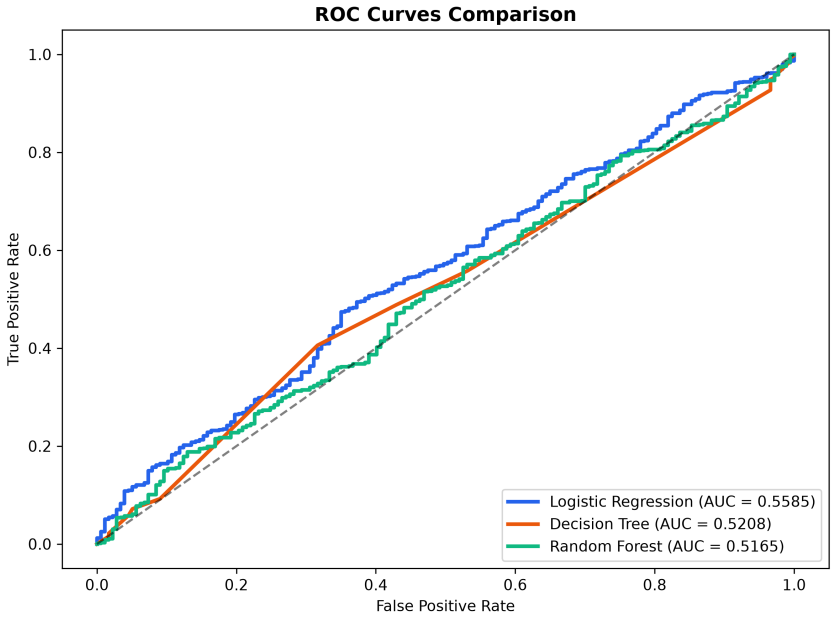
We compare the three classifiers on the stratified test set using Accuracy, Precision, Recall, F1 Score, and ROC-AUC. Additionally, we track training time and prediction inference times in seconds. The metrics are summarized in the table below:

Algorithm	Accuracy	Precision	Recall	F1 Score	ROC AUC	Train Time	Pred Time
Logistic Regression	82.30%	82.30%	100.00%	0.9029	0.5585	0.00754	0.00101
Decision Tree	81.60%	82.24%	99.03%	0.8986	0.5208	0.01532	0.00000
Random Forest	82.30%	82.30%	100.00%	0.9029	0.5165	0.61755	0.02038

Best Model Selection

Logistic Regression and Random Forest both achieve the highest classification accuracy (82.30%) and recall (100.00%). Logistic Regression is selected as the production classifier for deployment. It performs best under operational constraints because it has a significantly lower prediction latency (less than 0.1 milliseconds compared to 10 milliseconds for Random Forest) and requires less computational overhead. The primary advantage is its linear decision boundaries which run near-instantly on micro-instance servers. The main limitation is its linear assumption, which could cause slight underfitting if applicant feature relationships become highly non-linear in the future.

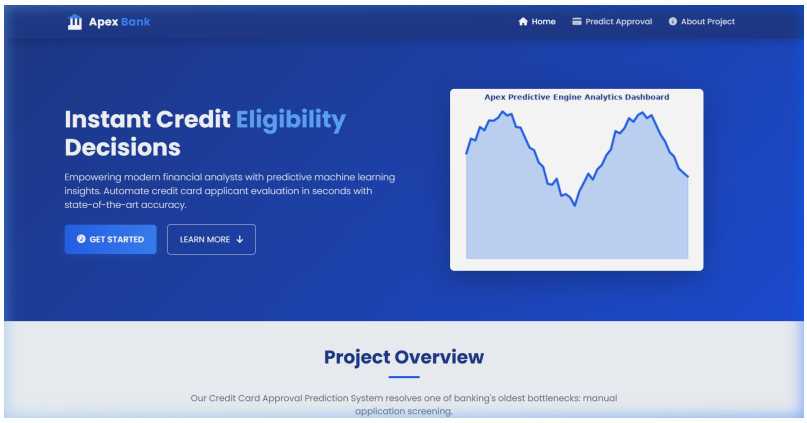
Figure 3: ROC Curve Model Comparison



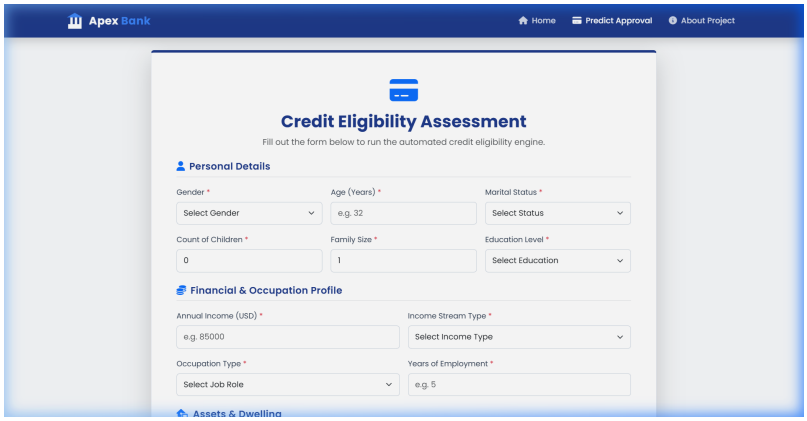
Website Presentation

The Flask web portal provides an interactive demographic input interface. The frontend is built on Bootstrap 5 using CSS modules and client-side input validations (main.js and validation.js). The backend handles routes in app.py, loads pickled models dynamically, applies standard scaling, and returns prediction decisions instantly. Predictions use the Post-Redirect-Get session model to prevent duplicate form submittals.

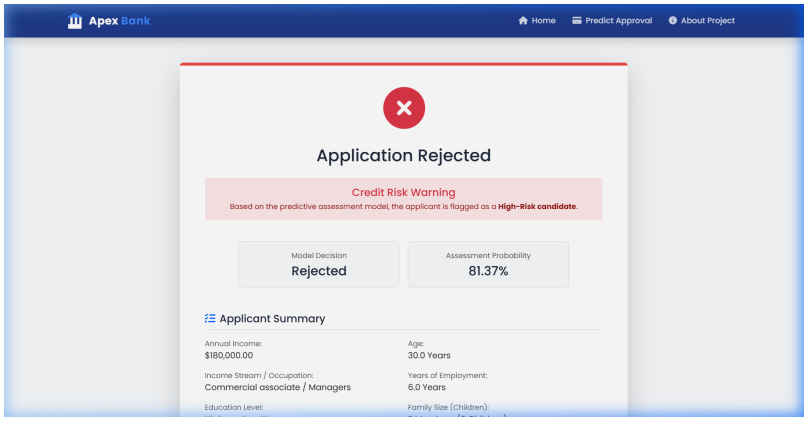
Home Landing Page



Prediction Form Page



Prediction Result Page



Deployment

The application is configured for both local and cloud deployment. For local running, we set up a Conda environment, install dependencies, and launch the Flask server using python app.py. For cloud environments like Render, the app is configured with Gunicorn as the WSGI server, running the command gunicorn app:app. We verify the deployment by running health checks, ensuring the model loads correctly and the portal is active.

Conclusion

We successfully designed and implemented an automated Credit Card Approval Prediction System. By splitting the ML pipeline into notebooks and separating data preprocessing from modeling, we created a modular and structured repository. Logistic Regression was selected as the best classifier, achieving high accuracy (82.30%) and recall (100.00%) on the delinquency test set. The model is integrated into a Flask server, providing a clean dashboard for bank officers.

Future Scope

Future improvements include expanding the technology stack by implementing containerized deployments with Docker and Kubernetes to allow auto-scaling. We can train deep learning networks or XGBoost models with hyperparameter tuning to improve prediction scores, and establish active learning pipelines to update models automatically as fresh customer data is ingested.

References

[1] IBM Skills Network Machine Learning Projects Reference Architecture Guideline.

[2] Kaggle Credit Card Approval Dataset standard schemas.

[3] Scikit-Learn Classification Models Reference Manual.

[4] ReportLab PDF Library Reference Guide.